Lab 2. Lakehouse end-to-end en AWS

Batch y streaming para PayTrack Colombia

Ceyni Paola Beltrán Hoyos y Carlos Guillermo Zúñiga Méndez

SI7006 Almacenamiento y Procesamiento de Grandes Datos - Maestría en Ingeniería de Datos

Profesor: Andrés Sacre Alzate - Universidad EAFIT

8 de noviembre de 2026 · Repositorio: github.com/caronte281197/big-data-labs-2026-2 (rama lab2)

1. PIPELINE BATCH CON MEDALLION Y DELTA LAKE (PARTE A)

Corrimos el pipeline batch primero en local con run_local.sh: Spark 3.5.3 y Delta Lake 3.2.1 sobre Docker, en un MacBook Pro con chip M5 Pro. Usamos los dos lotes del generador del curso. El lote 1 (ingest_date = 2026-09-29) trae 60 días de historia, con 30.600 filas de transacciones, 500 clientes, 8 comercios y 120 eventos de fraude. El lote 2 (2026-09-30) trae 2.316 filas de transacciones nuevas y de contracargos de transacciones del lote 1, 7 eventos de fraude y otra vez los 500 clientes, de los que 15 cambiaron de ciudad y 10 de segmento. Corrimos el lote 2 una segunda vez para probar la idempotencia y, al final, los dos lotes en AWS con Glue 5.0, donde dieron las mismas cifras. La Tabla 1 resume lo que dejó cada ejecución. En el repositorio están las capturas de cada salida, en labs/02-lakehouse/evidencias, y el diagrama de la arquitectura, en labs/02-lakehouse/diagrama.

Tabla 1. Resultado de cada ejecución del pipeline batch. La versión 0 de silver.transactions es la tabla vacía; en la repetición, Bronze y la cuarentena reescribieron la partición 2026-09-30 con las mismas filas.

Medida	Lote 1 (2026-09-29)	Lote 2 (2026-09-30)	Lote 2 repetido
Filas en raw y en Bronze	30.600	2.316	2.316
Transacciones distintas	30.000	2.276	2.276
Insertadas en silver.transactions	28.233	1.885	0
Actualizadas en silver.transactions (contracargos)	0	276	0
Cuarentena por invalid_amount	1.180	75	75
Cuarentena por missing_customer	587	40	40
Versión de silver.transactions	1	2	3
Filas reescritas por el MERGE (numTargetRowsCopied)	0	27.957	30.118

1.1 Bronze

Bronze lee todas las columnas como texto (primitivesAsString) porque los datos no llegan con un tipo estable: en el lote 1, 600 de los 30.600 montos vienen como texto ("45338.31") y el resto como número. Si Spark infiriera el esquema, amount_cop sería texto en un lote y número en otro, y Delta rechazaría el lote cuyo tipo no coincidiera con el de la tabla. Así, cada lote entra completo y el tipado queda para Silver, donde un monto que no se puede convertir queda nulo y va a cuarentena por invalid_amount.

La escritura usa overwrite con replaceWhere sobre ingest_date: cada ejecución reemplaza solo la partición de su lote, en un solo commit, sin tocar las demás fechas. Al repetir el lote 2,

la versión 2 de bronze/transactions retiró el archivo de la partición 2026-09-30 y escribió otra vez sus 2.316 filas; con append, la partición habría quedado con 4.632, cada transacción dos veces.

1.2 Silver

En el lote 1 quedaron en cuarentena 1.180 filas por invalid_amount y 587 por missing_customer, y en el lote 2, 75 y 40; ninguna por invalid_status (Tabla 1). Deduplicamos antes de las reglas para juzgar cada transacción una sola vez, sobre la copia que se queda. De las 600 transacciones repetidas del lote 1, en 58 una copia pasa las reglas y la otra no, por ejemplo una con 80.718,03 COP y otra con 0. Con las reglas primero, la copia buena iría a Silver y la mala a cuarentena, y la misma transacción quedaría en los dos lados. Deduplicando primero, las cuentas cuadran: 28.233 filas en Silver más 1.767 en cuarentena son las 30.000 transacciones distintas.

1.3 MERGE

La condición t.status <> s.status hace que el MERGE solo actualice lo que cambió. Al repetir el lote 2, sus 2.161 filas válidas ya estaban en Silver con el mismo status, y la versión 3 quedó con 0 insertadas y 0 actualizadas. Sin la condición, el MERGE habría actualizado esas 2.161 filas con los mismos valores: la auditoría leería 2.161 actualizadas, pero con time travel no encontraría ningún cambio de status, y su validación fallaría con 0 frente a 2.161. La condición evita cambios falsos, pero no las reescrituras: la repetición igual reescribió las 30.118 filas de la tabla, porque Delta reescribe completo cada archivo que contiene alguna llave del lote.

1.4 Time travel

La auditoría comparó las versiones 1 y 2 de silver.transactions con versionAsOf y encontró 276 transacciones que pasaron de approved a chargeback, por 20.943.086,12 COP, igual en local y en AWS. Según el TODO 4 de Athena, retail es la categoría con más monto revertido: 5.719.122,94 COP en 61 contracargos. Time travel funciona porque el MERGE no borra los archivos de la versión anterior, solo los marca como retirados en el _delta_log. Un VACUUM los borra cuando superan la retención, 7 días por defecto, y desde ese momento leer la versión 1 falla y la auditoría no tiene contra qué comparar. Por eso el resultado queda en gold.status_changes, aunque cada corrida la sobrescribe: al repetir el lote 2 en local quedó vacía.

1.5 PII

En el lote 2 cambiaron 25 clientes, 15 de ciudad y 10 de segmento. Con tipo 1, silver.customers solo guarda el valor nuevo y se pierde todo análisis con el atributo vigente en la fecha de la transacción: las cerca de 600 transacciones del lote 1 de quienes cambiaron de segmento ahora cuentan en el segmento nuevo, y gold.fraud_weekly_report reescribe semanas pasadas cada vez que un cliente cambia. En el Lab 1 lo resolvimos con dim_customer como SCD tipo 2: cada cambio cierra la fila vigente (valid_to, is_current) y abre otra, y cada hecho se une con

la versión vigente en su fecha. En Delta se hace con un MERGE que cierra la versión actual e inserta la nueva, con el email enmascarado en todas.

1.6 Consumo

El TODO 1 leyó 3.147 bytes de gold_daily_merchant_summary y el TODO 2, 96.946 de silver_transactions, unas 31 veces más, aunque los dos filtran por year y month: para septiembre, gold tiene unas 240 filas, una por día y comercio, y silver cerca de 16.000. Athena cobra mínimo 10 MB por consulta, así que hoy cuestan lo mismo; con 500.000 transacciones diarias, la consulta a silver leería 15 millones de filas al mes y la de gold seguiría en unas 240. Gold existe para las preguntas que se repiten, con la métrica calculada una sola vez. A cambio pierde detalle y frescura: los contracargos del streaming cayeron sobre transacciones de septiembre después de correr gold, y para M001 el TODO 2 da 98,19 % de aprobación mientras gold sigue en 99,04 %. La Tabla 2 resume los resultados y los bytes escaneados de las seis consultas.

Tabla 2. Consultas de Athena en AWS, con los bytes que escaneó cada una.

Consulta	Tablas	Bytes escaneados	Resultado principal
TODO 1	gold_daily_merchant_summary	3.147	M001 Éxito Laureles encabeza septiembre con 162.091.110,36 COP aprobados y 99,04 % de aprobación
TODO 2	silver_transactions	96.946	Los mismos cinco comercios, con M001 en 159.785.930,81 COP y 98,19 %, porque Silver ya tiene los contracargos del streaming
TODO 3	gold_fraud_weekly_report	356	Semana con más eventos por segmento: 24 de agosto en mass (14), 17 de agosto en high_value (4) y 14 de septiembre en premium (4)
TODO 4	gold_status_changes y silver_transactions	452.277	276 contracargos por 20.943.086,12 COP. Retail es la categoría con más monto: 61 contracargos y 5.719.122,94 COP
TODO 5	gold_fraud_alerts y bronze_transaction_events	271.862	524 alertas y 179 aciertos (34,2 %): 85,8 % a 20 eventos/s y 16,4 % a 50 eventos/s
TODO 6	gold_merchant_minute_metrics y silver_transactions	469.112	21.131 pagos del stream en Silver frente a 14.464 en la capa rápida

2. PIPELINE DE STREAMING SOBRE EL MISMO LAKEHOUSE (PARTE B)

Corrimos el streaming primero en local, con Kafka: una vez en modo availableNow, con 10 minutos de eventos a 20 eventos/s enviados de una vez (--fast), y dos veces en vivo, a 50 y a 20 eventos/s. En AWS, el job de Glue Streaming leyó de Kinesis en dos corridas de 5 minutos, a 20 y a 50 eventos/s. La Tabla 3 resume las corridas. En AWS la precisión sale del TODO 5 de Athena; en local la calculamos con la misma regla en data/analisis/experimentos_stream.py.

Nota: la primera corrida de streaming en AWS se detuvo al arrancar. Al revisar el registro del job y la documentación de AWS encontramos la causa: la versión de Glue que usamos entrega la hora de llegada de cada evento con un nombre de columna distinto al que espera jobs/common.py, uno de los archivos dados del laboratorio. Hicimos en ese archivo un ajuste pequeño, que no cambia los resultados, y lo dejamos explicado en un comentario; con él, las

dos corridas siguientes terminaron bien. Los eventos de ese primer intento no se perdieron: quedaron en Kinesis y el job los procesó en la corrida a 20 eventos/s.

Tabla 3. Corridas de streaming. Descartadas: filas que la consulta metrics descartó por watermark. Precisión: porcentaje de alertas en las que el cliente tiene un pago fraudulento dentro de la ventana (regla del TODO 5).

Corrida	Eventos	Descartadas por watermark	Alertas	Aciertos	Precisión
Local, availableNow (--fast)	12.227	3 (0,02 %)	131	100	76,3 %
Local, en vivo a 50 eventos/s	6.177	103 (1,7 %)	388	98	25,3 %
Local, en vivo a 20 eventos/s	6.147	120 (2,0 %)	38	30	78,9 %
AWS, en vivo a 20 eventos/s (incluye el intento fallido)	9.878	65 (0,7 %)	134	115	85,8 %
AWS, en vivo a 50 eventos/s	12.299	151 (1,2 %)	390	64	16,4 %

2.1 Garantías de entrega

El productor es at-least-once. En Kafka usa acks=all y la idempotencia del productor, así que un reintento no duplica mensajes, pero el productor reenvía a propósito el 3 % de los eventos con el mismo event_id. En Kinesis, put_records puede aceptar unos registros y rechazar otros; reintentamos solo los rechazados, y si se pierde una respuesta un registro puede quedar dos veces. Las tablas quedan como si cada evento se procesara una vez porque el checkpoint guarda hasta dónde se leyó, Delta escribe cada micro-lote en una transacción, y el MERGE por transaction_id y la deduplicación por event_id absorben las repeticiones. Al correr otra vez el job en availableNow leyó 0 filas, y las tablas siguieron con 41.740 transacciones, 92 ventanas y 98 alertas.

Nota: al conectarse con Kafka, el productor mostró un aviso de reintento que se resolvió solo, sin perder eventos.

2.2 Watermark (experimento 1)

En availableNow, metrics y alerts descartaron 3 de 12.227 eventos; en vivo, 103 a 50 eventos/s (1,7 %) y 120 a 20 eventos/s (2,0 %) (Tabla 3). La diferencia es cuándo avanza el watermark: en availableNow el job leyó todo en pocos micro-lotes y los tardíos llegaron junto con los demás, antes de que el watermark los dejara atrás; en vivo el watermark va unos 2 minutos detrás del reloj y un evento con más de 2 minutos de retraso encuentra su ventana cerrada. Cuadra con el productor: 3 % de tardíos con retrasos de 30 a 360 segundos, de los que el 73 % pasan de 2 minutos, da 2,2 %. En AWS fue menos (0,7 y 1,2 %) porque cada job arrancó con un minuto o más de eventos en cola y los procesó de una vez. Silver no usa watermark porque es la fuente de verdad: con watermark perdería esos pagos tardíos.

2.3 Latencia

En la corrida local a 20 eventos/s detectamos 13 ráfagas, y entre el último pago de cada una y su primera alerta en gold.fraud_alerts pasaron 155 segundos de mediana. En promedio fueron 14 s hasta el cierre de la ventana, 120 s de watermark y 22 s de micro-lotes, así que el modo append con el watermark pone 134 de los 156 segundos (86 %): una ventana solo se emite cuando el watermark pasa su final. Para bajarla se puede acortar el watermark, a costa

de descartar más tardíos, o emitir en modo update hacia una tabla que admita actualizar la alerta, a costa de alertas que cambian después de escritas.

2.4 Precisión (experimento 2)

A 20 eventos/s la precisión fue de 76,3 % y 78,9 % en local y de 85,8 % en AWS; a 50 eventos/s cayó a 25,3 % y 16,4 % (Tabla 3). La regla alerta cuando un cliente hace 6 o más pagos en una ventana de 30 segundos, y con 500 clientes el azar depende del ritmo: a 20 eventos/s cada cliente hace en promedio 1,1 pagos por ventana y llegar a 6 tiene una probabilidad de 0,1 %; a 50 eventos/s el promedio sube a 2,8 y la probabilidad a cerca de 7 %. Como una ráfaga real son de 6 a 8 pagos separados 2 segundos, ajustaríamos la ventana a 15 segundos, con un avance de 5: a 50 eventos/s el promedio bajaría a 1,4 y la probabilidad a 0,3 %.

2.5 Estado final

Sin la condición t.status <> 'chargeback', un reenvío revertiría contracargos. El productor reenvía, en el 3 % de los casos, uno de sus últimos 200 eventos con el mismo event_id, y ese evento puede ser el pago original de una transacción cuyo contracargo ya llegó a Silver. El reenvío trae status approved y llega en un micro-lote posterior, así que el MERGE devolvería la transacción a aprobada. latest_per_transaction no lo evita porque solo compara los eventos de un mismo micro-lote, y el contracargo quedó en Silver en un micro-lote anterior. La validación "silver: ningún contracargo fue revertido por un reenvío" pasó en local y en AWS.

2.6 Partition key (experimento 3)

Con llave constante, los 1.232 eventos de la prueba cayeron en una sola partición y las otras dos quedaron vacías; con transaction_id, en una prueba igual, quedaron 396, 407 y 422. Con customer_id, la llave del productor, el topic principal quedó con 9.228, 7.448 y 7.875 eventos, en proporción a los clientes de cada partición (190, 151 y 159). Con transaction_id Spark detectaría las mismas alertas, porque agrupa por cliente y las ventanas usan event_time. La llave sirve para el orden: Kafka y Kinesis solo lo garantizan dentro de una partición, y con la misma llave el pago y su contracargo caen en la misma partición y se leen en el orden en que se enviaron, así que Silver no ve el contracargo antes que su pago.

2.7 Batch y streaming en la misma tabla

En AWS, silver.transactions terminó con 32 versiones: 3 del batch y 29 MERGE del streaming, 15 en la corrida a 20 eventos/s y 14 en la de 50. Cada MERGE del stream reescribió en mediana unas 38.000 filas (numTargetRowsCopied); en total copió 1.020.020 para insertar 21.133 y actualizar 399, unas 47 filas reescritas por cada fila que cambió, porque Delta reescribe completo cada archivo que contiene una llave del micro-lote. A 500.000 pagos diarios serían 8.640 versiones por día, y cada una reescribiría buena parte de la tabla. Lo resolveríamos por dos lados: menos MERGE, con micro-lotes de uno a cinco minutos en vez de 10 segundos, y MERGE más baratos, con la tabla agrupada por transaction_id (clustering) para que cada uno toque pocos archivos, o con deletion vectors, que marcan las filas cambiadas en vez de reescribir el archivo. Si el job silver batch y el stream escribieran a la vez,

serían dos drivers en la misma tabla de S3, y la documentación de Delta advierte que eso puede perder datos porque S3 no da exclusión mutua para los commits; habría que usar el LogStore con DynamoDB, con el que uno de los dos MERGE falla por conflicto y se reintenta, o programarlos para que no se traslapen. Con el TODO 6, la fuente de verdad es Silver: tiene 21.131 transacciones del stream y la capa rápida cuenta 14.464. La diferencia son las ventanas de los últimos dos minutos de la corrida a 50, que nunca se cerraron, y las 216 filas que metrics descartó por watermark.

3. COSTOS Y OPERACIÓN

La ejecución completa en AWS costó cerca de medio dólar, casi todo de Glue. Para 500.000 transacciones diarias estimamos unos 794 USD al mes (Tabla 4). Glue Streaming encendido todo el mes es el 80 % de ese total, y le siguen las solicitudes a S3 del streaming, con el 16 %: cada micro-lote escribe y lista checkpoints, estado y archivos Delta, y con un micro-lote cada 10 segundos son 8.640 al día.

Tabla 4. Costo mensual estimado con 500.000 transacciones diarias, con precios de lista de AWS para US East y un mes de 720 horas.

Componente	Supuesto	USD al mes
Glue Streaming 24/7	2 workers G.1X (2 DPU) todo el mes, a 0,44 USD por DPU-hora	633,60
S3, solicitudes	8.640 micro-lotes al día con unas 100 escrituras y listados cada uno (en local, 64 archivos nuevos por micro-lote), a 0,005 USD por mil	129,60
Kinesis	2 shards a 0,015 USD por hora y 15,8 millones de registros al mes, a 0,014 USD por millón	21,82
Glue batch	4 jobs con 2 DPU, unos 20 minutos al día (hoy tardan unos 7 por lote)	8,80
S3, almacenamiento	unos 8 GB nuevos al mes entre raw y las tablas Delta, a 0,023 USD por GB	0,19 (y crece)
Athena	3.000 consultas al mes a gold, cobradas al mínimo de 10 MB, a 5 USD por TB	0,15
Total		794,16

Si el streaming no necesitara correr 24/7, lanzaríamos el mismo job cada hora en modo availableNow (RUN_SECONDS = 0): Kinesis guarda los eventos 24 horas, y el job procesa lo acumulado y termina. Con unos 3 minutos por corrida, Glue Streaming bajaría de 633,60 a unos 32 USD al mes y las solicitudes a S3 casi desaparecerían, a cambio de alertas con hasta una hora de retraso. Si tiene que seguir 24/7, un micro-lote por minuto en vez de cada 10 segundos divide por seis las solicitudes a S3 y los MERGE de la sección 2.7.

No se debe poner una regla de expiración sobre lakehouse/ porque S3 borra por edad, sin mirar el _delta_log. Los archivos de una partición de agosto que nadie vuelve a modificar siguen siendo parte de la versión actual, y al vencer, las consultas fallarían por archivos que ya no existen; si vencen los archivos del _delta_log de una tabla que no se ha escrito en ese tiempo, la tabla deja de poder leerse. Lo que sobra en lakehouse/ se limpia con VACUUM, que solo borra archivos que ninguna versión vigente usa y que superaron la retención. Para raw/ sí dejamos una regla en serverless.yml: pasa cada lote a Standard-IA a los 30 días y a Glacier

Instant Retrieval a los 90, y lo borra a los 10 años, el plazo que da la Ley 962 de 2005 (art. 28) para conservar los soportes contables.

Nota: la cuenta de AWS que abrimos para el laboratorio tenía bloqueados los jobs de Glue, y el primer despliegue no pudo crearlos. Lo consultamos con el soporte de AWS, que llevó el caso al equipo de Glue, y el servicio quedó habilitado. En general recomendamos la experiencia: los dos inconvenientes que tuvimos se resolvieron investigando en la documentación y en los registros de los jobs, y con el soporte de AWS cuando hizo falta.